

```

//
=====
=====
// LL_MQ_CO_GC_LO_1_0 .ino
//
// NGR Lowline — Manual MQTT + CTO/CE Dispatcher-Control Combo
// Flash to Otto (9950011) or Toby (9950012) using LL_LocoConfig.h
//
// BASE:
// LL_CE_GC_Loco_v1_2_candidate_escape.ino
//
// CHANGES FROM v1.2 — v1.3
// -----
// 1. CE exit spacing correction:
//   When CE Local exits Circuit Express and becomes the new CTO leader,
//   its FIRST post-CE leader stop uses the existing CE Local ramp-delay
//   distance before braking. This keeps the visually credible smooth
//   handoff at Local speed while moving the new leader farther up the
//   station, leaving more room for the former Express / new trailer.
//
//   Scope is intentionally narrow:
//   - one first-stop approach-delay override only
//   - ordinary CTO leader hold/release behavior remains unchanged
//   - ordinary CTO leader behavior resumes after that first stop
//
// 2. True limited-stop Express behavior:
//   The Express now bypasses two station detections, stops at the third
//   for the ordinary 5-second station dwell, then repeats:
//   skip, skip, stop; skip, skip, stop...
//
//   This preserves CE's catch-up behavior while making the Express read
//   as a real limited-stop train rather than a nonstop train.
//
// EXISTING v1.2 FEATURES RETAINED:
// - CE begins only from valid CTO order state already established by CTO.
// - CTO leader -> Express; CTO trailer -> Local.
// - CE Express PWM 120; CE Local PWM 80.
// - CE Local station-service behavior preserved.
// - CE -> CTO failed-recapture trailing-candidate recovery preserved.
// - Passive CE Local -> CTO Leader diagnostic STATUS packet preserved.
//
// PROTOCOL NOTE:
// VERSION remains 14. Packet structure is unchanged and remains compatible
// with LL_CE_GC_Dispatcher_v1_2.
//
=====

```

```

=====

#include "LL_LocoConfig.h"
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
#include <Wire.h>
#include <Adafruit_INA219.h>
#include <PubSubClient.h>

//
=====
=====
// PROTOCOL CONSTANTS
//
=====
=====
static const uint8_t MAGIC = 0x4E;
static const uint8_t VERSION = 14;    // Protocol unchanged from CE GC v1.2

//
=====
=====
// HARDWARE CONSTANTS
//
=====
=====
static const int HALL_PIN = 33;
static const int ADC_SAMPLES = 12;
static const unsigned long SAMPLE_INTERVAL_MS = 40UL;
static const unsigned long SECOND_MAGNET_TIMEOUT_MS = 5000UL;
static const unsigned long PAIR_LOCKOUT_MS = 1500UL;
static const unsigned long CALIBRATION_TIME_MS = 2000UL;
static const int DEFAULT_DEADBAND_COUNTS = 120;
static const int TRIGGER_MARGIN_COUNTS = 40;

static const int STOPPED_PWM_THRESHOLD = 25;
static const unsigned long STOPPED_DEBOUNCE_MS = 250UL;

static const uint8_t BROADCAST_MAC[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};

//
=====
=====
// MQTT / MANUAL-WEB INTEGRATION
// Pi-side Flask app publishes commands; loco publishes authoritative state.

```

```

// The ESP32 connects to the Pi broker over WiFi.
//
=====
=====
#define MQTT_BROKER "192.168.68.142"
#define MQTT_PORT 1883
#define MQTT_CLIENT_ID LOCO_NAME

static char TOPIC_ONLINE[56];

static char TOPIC_CMD_THROTTLE[56];
static char TOPIC_CMD_DIRECTION[56];
static char TOPIC_CMD_BRAKE[56];
static char TOPIC_CMD_ESTOP[56];
static char TOPIC_CMD_AUTO[56];
static char TOPIC_CMD_DISPATCHER_RELEASE[56];

static char TOPIC_STATE_THROTTLE[56];
static char TOPIC_STATE_DIRECTION[56];
static char TOPIC_STATE_BRAKE[56];
static char TOPIC_STATE_ESTOP[56];
static char TOPIC_STATE_AUTO[56];
static char TOPIC_STATE_LOWVOLT[56];
static char TOPIC_STATE_WARNING[56];
static char TOPIC_STATE_BLOCK[56];

static char TOPIC_TELEM_VOLTAGE[56];
static char TOPIC_TELEM_CURRENT[56];
static char TOPIC_TELEM_POWER[56];

WiFiClient mqttWifiClient;
PubSubClient mqtt(mqttWifiClient);

//
=====
=====
// INA219 TELEMETRY / LOW-VOLTAGE PROTECTION
// This is the GoldCore v1.7 safety/telemetry layer, ported into the combo
// sketch so the Flask dashboards retain voltage/current/power and low-voltage
// status. It is independent of CTO/CE packet logic.
//
=====
=====
#define INA_SDA_GPIO 21
#define INA_SCL_GPIO 22

```

```

//
=====
=====
// FRESHNESS THRESHOLDS
//
=====
=====
static const unsigned long LOGIC_FRESH_LIMIT_MS = 45000UL;
static const unsigned long HEALTH_WARN_LIMIT_MS = 120000UL;
static const unsigned long HEALTH_STOP_LIMIT_MS = 240000UL;

//
=====
=====
// CIRCUIT EXPRESS — operational profile overlay
// CE structure follows the working v7 model, added onto v2.1 GoldCore CTO.
// CTO remains the normal operating mode.
//
=====
=====
static const int      CE_EXPRESS_PWM    = 120;
static const int      CE_LOCAL_PWM      = 80;
static const unsigned long CE_LOCAL_DWELL_MS = 5000UL;

// Express limited-stop service:
// bypass two station detections, stop at the third, repeat.
static const uint8_t  CE_EXPRESS_STOP_EVERY_N_STATIONS = 3;

// CE -> CTO failed-recapture escape.
// When the former Express stops short as a provisional trailer but no CTO
// pairing completes, it must not remain gridlocked forever.
static const int      CE_TRAILING_CANDIDATE_CAUTION_PWM = 75;
static const unsigned long CE_TRAILING_CANDIDATE_ESCAPE_MS =
CE_LOCAL_DWELL_MS + 5000UL;

//
=====
=====
// MESSAGE TYPES
//
=====
=====
enum MsgType : uint8_t {
    MSG_TYPE_BLOCK      = 1,
    MSG_TYPE_ESTOP      = 3,
    MSG_TYPE_MANUAL      = 4,
    MSG_TYPE_ENCOUNTER_START = 5,

```

```

MSG_TYPE_RELEASE      = 6,
MSG_TYPE_DEPARTING    = 7,
MSG_TYPE_STATUS       = 8
};

//
=====
=====
// STATUS EVENT CODES
// Carried in the cmd field of MSG_TYPE_STATUS packets.
// The Dispatcher logs these by name. The value field carries current PWM.
// The flags field carries bit0=confirmedStopped, bit1=departingReceived.
//
=====
=====
enum StatusEvent : uint8_t {
    ST_BLOCK_DETECTED      = 1,
    ST_ROLE_ASSIGNED       = 2,
    ST_RAMP_DELAY_START    = 3,
    ST BRAKING              = 4,
    ST_CONFIRMED_STOP      = 5,
    ST_RELEASE_SENT        = 6,
    ST_WAIT_LEADER         = 7,
    ST_ADVANCE_START       = 8,
    ST_ADVANCE_END         = 9,
    ST_PLATFORM_STOP       = 10,
    ST_PLATFORM_DWELL      = 11,
    ST_DEPARTING           = 12,
    ST_LEADER_WAITING      = 13,
    ST_LEADER_RELEASED     = 14,
    ST_CE_LOCAL_EXIT_TO_CTO_LEADER = 15
};

//
=====
=====
// HALL / DETECT / BLOCK / CMD ENUMS
//
=====
=====
enum HallState { HALL_NONE, HALL_NORTH, HALL_SOUTH };
enum DetectState { DETECT_RUNNING, DETECT_WAIT_SECOND,
DETECT_DWELLING };

enum BlockId : uint8_t {
    BLOCK_UNKNOWN = 0,
    BLOCK_PATIO   = 1,

```

```
BLOCK_BAMBOO = 2,  
BLOCK_ARCHES = 3,  
BLOCK_GRILLERS = 4  
};
```

```
enum CmdType : uint8_t {  
    CMD_NONE = 0,  
    CMD_START = 1,  
    CMD_STOP = 2,  
    CMD_ESTOP = 3,  
    CMD_SET_PWM = 10,  
    CMD_SET_ROLE_PARAM = 14,  
    CMD_CLEAR_ALL = 13,  
    CMD_CE_START = 20  
};
```

```
static const uint8_t ROLE_ID_DEFAULT = 0;  
static const uint8_t ROLE_ID_STATION = 1;  
static const uint8_t ROLE_ID_TRAILING = 2;  
static const uint8_t ROLE_ID_LEADER = 3;
```

```
static const uint8_t PARAM_RAMP_DELAY = 1;  
static const uint8_t PARAM_RAMP_RATE = 2;  
static const uint8_t PARAM_DWELL = 3;  
static const uint8_t PARAM_FOLLOW_DELAY = 4;  
static const uint8_t PARAM_ADVANCE_MS = 5;  
static const uint8_t PARAM_PLATFORM_DWELL = 6;  
static const uint8_t PARAM_RAMP_UP_RATE = 7;  
static const uint8_t PARAM_ADVANCE_PWM = 8;
```

```
enum StopReason : uint8_t {  
    STOP_NONE = 0,  
    STOP_STATION_DWELL = 1,  
    STOP_CTO_TRAILING = 2,  
    STOP_CTO_LEADER_HOLD = 3,  
    STOP_ESTOP = 4  
};
```

```
enum EncounterRole : uint8_t {  
    ROLE_NONE = 0,  
    ROLE_TRAILING = 1,  
    ROLE_LEADING = 2  
};
```

```
enum CeRole : uint8_t {  
    CE_NONE = 0,  
    CE_EXPRESS = 1,
```

```

    CE_LOCAL = 2
};

enum TrailingSubState : uint8_t {
    TR_IDLE = 0,
    TR_WAIT_LEADER_DEPART = 1,
    TR_FOLLOW_DELAY = 2,
    TR_ADVANCE_TO_PLATFORM = 3,
    TR_PLATFORM_STOP = 4,
    TR_PLATFORM_DWELL = 5
};

//
=====
=====
// PACKET STRUCTURE
//
=====
=====
typedef struct __attribute__((packed)) {
    uint8_t magic;
    uint8_t version;
    uint8_t msgType;
    uint32_t senderId;
    uint32_t locId;
    uint8_t blockId;
    uint8_t cmd;
    uint8_t flags;
    uint32_t sequenceNumber;
    uint32_t encounterId;
    uint16_t value;
    uint8_t roleId;
    uint8_t paramId;
} TrainPacket;

//
=====
=====
// ROLE PROFILES
//
=====
=====
struct RoleProfile {
    unsigned long rampDelayMs;
    unsigned long rampRateMs;
    unsigned long dwellMs;
};

```

```

struct TrailingParams {
    unsigned long followDelayMs;
    unsigned long advanceMs;
    unsigned long platformDwellMs;
};

//
=====
=====
// COMPILED DEFAULTS
//
=====
=====
RoleProfile profileDefault = { 6000UL, 80UL, 2000UL };
RoleProfile profileStation = { 6000UL, 80UL, 5000UL };
RoleProfile profileTrailing = { 0UL, 53UL, 500UL };
RoleProfile profileLeader = { 5000UL, 80UL, 4000UL };
RoleProfile profileCeLocal = { 12000UL, 80UL, CE_LOCAL_DWELL_MS };
RoleProfile profileCeExpressStop = { 4500UL, 80UL, 5000UL };

TrailingParams trailingParams = {
    3000UL, // followDelayMs
    15000UL, // advanceMs
    10000UL // platformDwellMs
};

unsigned long rampUpRateMs = 120UL;
int trailingAdvancePwmOverride = 0;

//
=====
=====
// PROFILE SELECTOR
//
=====
=====
RoleProfile& profileForRole(uint8_t roleId) {
    switch (roleId) {
        case ROLE_ID_STATION: return profileStation;
        case ROLE_ID_TRAILING: return profileTrailing;
        case ROLE_ID_LEADER: return profileLeader;
        default: return profileDefault;
    }
}

//

```



```

=====
=====
// PEER TABLE
//
=====
=====
static const int MAX_PEERS = 4;

struct PeerState {
    uint32_t locId;
    uint8_t lastBlock;
    uint32_t lastSeq;
    unsigned long lastSeen;
};

PeerState peers[MAX_PEERS];

//
=====
=====
// FUNCTION PROTOTYPES
//
=====
=====
bool peerIsLogicFresh(const PeerState& p);
const char* blockName(BlockId b);
void sendBlockReport(BlockId b);
void sendEncounterStart(uint32_t leaderId, uint8_t blockId, uint32_t encounterId);
void sendRelease(uint32_t leaderId, uint8_t blockId, uint32_t encounterId);
void sendDeparting(uint32_t targetId, uint8_t blockId, uint32_t encounterId);
void sendEstopAck();
void sendStatus(StatusEvent ev);
void sendCeLocalExitToCtoLeaderStatus();
void resetEncounterState();
void clearStickyLeaderDeparture();
BlockId nextBlock(BlockId b);
int blocksAhead(uint8_t fromBlock, uint8_t toBlock);
bool leaderDepartureWithinTwoBlocks(uint8_t leaderBlock);
void breakTrailingRelationshipIfTooFar(uint8_t leaderBlock);
void applyStickyLeaderDepartureIfMatching();
void applyPendingEncounterIfMatching(uint32_t trailingId, uint8_t blockId);
void clearCtoToFreshRunningState();
bool getFreshPeerBlock(uint8_t* outBlock);
void startCircuitExpressIfEligible();
void stopCircuitExpressStateOnly();
void updateCeExitArmedFromGeometry();
bool tryCeReturnToCtoAtBlock(BlockId b);

```

```

RoleProfile& activeProfile();
unsigned long activeRampDelayMs();
bool serviceStopDebounce();
void setSharedRampRate(unsigned long rateMs);
void mqttSetupTopics();
void mqttPublishThrottle();
void mqttPublishDirection();
void mqttPublishBrake();
void mqttPublishEstop();
void mqttPublishAuto();
void mqttPublishLowvolt();
void mqttPublishWarning(const char* msg);
void mqttPublishTelemetry(float busVoltage);
void mqttPublishBlock();
void mqttPublishFullStateSnapshot();
void onMQTTMessage(char* topic, byte* payload, unsigned int length);
void serviceMQTTReconnect();
void serviceINA219();
void applyDirectionPin();
void enterDispatcherAutoMode();
void leaveDispatcherAutoMode();
void clearCeTrailingCandidateState();

```

```
//
```

```
=====
```

```
=====
```

```
// RUNTIME VARIABLES
```

```
//
```

```
=====
```

```
=====
```

```

int cruisePwm  = NORMAL_PWM;
int rampCurrent = 0;
int rampTarget = 0;
unsigned long lastRamp  = 0;
bool          running   = false;
unsigned long lastBlockTime = 0;
StopReason    stopReason = STOP_NONE;
uint8_t       myCurrentBlock = BLOCK_UNKNOWN;
uint32_t      txSequence     = 0;
uint32_t      localEncounterCounter = 0;

```

```
//
```

```
=====
```

```
=====
```

```
// Manual web-control / MQTT authority state
```

```
//
```

```
=====
=====
```

```
int direction    = DIRECTION_FORWARD; // 0=reverse, 1=neutral, 2=forward
int brakeValue   = 0;                 // manual deceleration style slider
bool eStopEngaged = false;
bool dispatcherAuto = false;          // true = Dispatcher owns the loco
```

```
// MQTT warning text is non-retained and clears itself after a short TTL.
static unsigned long mqttWarningClearAtMs = 0;
static const unsigned long MQTT_WARNING_TTL_MS = 2500UL;
```

```
// INA219 / low-voltage state
Adafruit_INA219 ina219;
bool ina219Present = false;
float current_mA    = 0.0f;
float power_mW      = 0.0f;
float lastBusVoltage = 0.0f;
```

```
int lowVoltageCounter    = 0;
int shutdownCounter      = 0;
bool lowVoltageThrottleActive = false;
bool voltageShutdownActive = false;
```

```
enum VoltageState : uint8_t { DISCONNECTED, LOW_VOLTAGE,
NORMAL_OPERATION };
VoltageState voltageState = DISCONNECTED;
VoltageState lastVoltageState = DISCONNECTED;
```

```
// Hall calibration
int baselineCounts    = -1;
int northEnterThreshold = -1;
int northExitThreshold = -1;
int southEnterThreshold = -1;
int southExitThreshold = -1;
```

```
// Block detection state
DetectState detectState = DETECT_RUNNING;
HallState currentHall    = HALL_NONE;
HallState firstMagnet    = HALL_NONE;
bool waitingForClear = false;
unsigned long lastSample = 0;
unsigned long stateStart = 0;
unsigned long lockoutUntil = 0;
bool rampDelayActive = false;
unsigned long rampDelayStart = 0;
bool stoppedDebounceActive = false;
unsigned long stoppedDebounceStart = 0;
```

```

bool    confirmedStopped    = false;

// CTO encounter state
EncounterRole encounterRole = ROLE_NONE;

bool    leaderHoldActive    = false;
bool    leaderReleaseReceived = false;
unsigned long leaderHoldStartedAt = 0;
uint32_t holdingForTrailingId = 0;
uint32_t holdingEncounterId  = 0;
uint8_t holdingBlock         = BLOCK_UNKNOWN;

bool    pendingStartActive    = false;
uint32_t pendingStartTrailingId = 0;
uint32_t pendingStartEncounterId = 0;
uint8_t pendingStartBlock     = BLOCK_UNKNOWN;

bool    pendingReleaseActive    = false;
uint32_t pendingReleaseTrailingId = 0;
uint32_t pendingReleaseEncounterId = 0;
uint8_t pendingReleaseBlock     = BLOCK_UNKNOWN;

bool    trailingEncounterActive = false;
bool    trailingReleaseSent     = false;
uint32_t trailingLeaderId      = 0;
uint32_t trailingEncounterId   = 0;
uint8_t trailingEncounterBlock = BLOCK_UNKNOWN;

TrailingSubState trailingSubState = TR_IDLE;
bool    departingReceived    = false;
bool    leaderReachedNextBlock = false;
unsigned long trailingTimerStart = 0;

// Sticky leader departure cache.
bool    stickyDepartingActive    = false;
uint32_t stickyDepartingLeaderId = 0;
uint8_t stickyDepartingBlock     = BLOCK_UNKNOWN;
uint32_t stickyDepartingEncounterId = 0;

// Circuit Express state.
// ctoOrderRole is remembered CTO order knowledge from the last ordinary CTO role
// assignment. It is used only when accepting CE.
CeRole ctoOrderRole = CE_NONE;
bool    ceActive    = false;
CeRole ceRole       = CE_NONE;
bool    ceExitArmed = false;
int     ceSavedCruisePwm = NORMAL_PWM;

```

```

// Express limited-stop service counter.
// Counts CE Express station detections after CE begins.
// 1 = skip, 2 = skip, 3 = stop, then reset.
uint8_t ceExpressStationCounter = 0;

// One-time first-stop spacing correction after CE Local becomes CTO leader.
// When true, the first post-CE CTO leader stop uses profileCeLocal.rampDelayMs
// before braking, then the flag clears.
bool postCeLeaderFirstStopDelayActive = false;

// Temporary state used only after CE Express reaches the Local's current
// block and attempts ordinary CTO recapture as a trailing locomotive.
bool      ceTrailingCandidateActive    = false;
uint32_t  ceTrailingCandidateLeaderId = 0;
uint8_t   ceTrailingCandidateBlock     = BLOCK_UNKNOWN;
unsigned long ceTrailingCandidateTimerStart = 0;

//
=====
=====
// ACTIVE PROFILE SELECTOR
//
=====
=====
RoleProfile& activeProfile() {
    if (ceActive && ceRole == CE_LOCAL && stopReason == STOP_STATION_DWELL) {
        return profileCeLocal;
    }

    switch (stopReason) {
        case STOP_STATION_DWELL: return profileStation;
        case STOP_CTO_TRAILING:  return profileTrailing;
        case STOP_CTO_LEADER_HOLD: return profileLeader;
        default:                  return profileDefault;
    }
}

// Ramp delay accessor.
// Ordinary behavior uses activeProfile().rampDelayMs.
// The only override is the first post-CE CTO leader stop, which borrows the
// existing CE Local approach delay to improve consist clearance.
unsigned long activeRampDelayMs() {
    if (postCeLeaderFirstStopDelayActive &&
        stopReason == STOP_CTO_LEADER_HOLD) {
        return profileCeLocal.rampDelayMs;
    }
}

```

```
    return activeProfile().rampDelayMs;
}
```

```
void setSharedRampRate(unsigned long rateMs) {
    profileDefault.rampRateMs = rateMs;
    profileStation.rampRateMs = rateMs;
    profileTrailing.rampRateMs = rateMs;
    profileLeader.rampRateMs = rateMs;
    Serial.printf("CMD: SHARED rampRateMs = %lu ms\n", rateMs);
}
```

```
//
```

```
=====
```

```
=====
```

```
// MQTT TOPICS / STATE PUBLISH
```

```
//
```

```
=====
```

```
=====
```

```
void mqttSetupTopics() {
```

```
    const char* id = LOCO_NAME;
```

```
    snprintf(TOPIC_ONLINE,      sizeof(TOPIC_ONLINE),      "ngr/loco/%s/status/
online", id);
```

```
    snprintf(TOPIC_CMD_THROTTLE,  sizeof(TOPIC_CMD_THROTTLE),  "ngr/loco/
%s/cmd/throttle", id);
```

```
    snprintf(TOPIC_CMD_DIRECTION, sizeof(TOPIC_CMD_DIRECTION), "ngr/loco/
%s/cmd/direction", id);
```

```
    snprintf(TOPIC_CMD_BRAKE,      sizeof(TOPIC_CMD_BRAKE),      "ngr/loco/%s/
cmd/brake", id);
```

```
    snprintf(TOPIC_CMD_ESTOP,      sizeof(TOPIC_CMD_ESTOP),      "ngr/loco/%s/
cmd/estop", id);
```

```
    snprintf(TOPIC_CMD_AUTO,        sizeof(TOPIC_CMD_AUTO),        "ngr/loco/%s/cmd/
auto", id);
```

```
    snprintf(TOPIC_CMD_DISPATCHER_RELEASE,
sizeof(TOPIC_CMD_DISPATCHER_RELEASE), "ngr/loco/%s/cmd/dispatcher_release",
id);
```

```
    snprintf(TOPIC_STATE_THROTTLE, sizeof(TOPIC_STATE_THROTTLE), "ngr/loco/
%s/state/throttle", id);
```

```
    snprintf(TOPIC_STATE_DIRECTION, sizeof(TOPIC_STATE_DIRECTION), "ngr/loco/
%s/state/direction", id);
```

```
    snprintf(TOPIC_STATE_BRAKE,      sizeof(TOPIC_STATE_BRAKE),      "ngr/loco/%s/
state/brake", id);
```

```
    snprintf(TOPIC_STATE_ESTOP,      sizeof(TOPIC_STATE_ESTOP),      "ngr/loco/%s/
```

```

state/estop", id);
    snprintf(TOPIC_STATE_AUTO, sizeof(TOPIC_STATE_AUTO), "ngr/loco/%s/
state/auto", id);
    snprintf(TOPIC_STATE_LOWVOLT, sizeof(TOPIC_STATE_LOWVOLT), "ngr/loco/
%s/state/lowvolt", id);
    snprintf(TOPIC_STATE_WARNING, sizeof(TOPIC_STATE_WARNING), "ngr/loco/
%s/state/warning", id);
    snprintf(TOPIC_STATE_BLOCK, sizeof(TOPIC_STATE_BLOCK), "ngr/loco/%s/
state/block", id);

    snprintf(TOPIC_TELEM_VOLTAGE, sizeof(TOPIC_TELEM_VOLTAGE), "ngr/loco/
%s/telem/voltage", id);
    snprintf(TOPIC_TELEM_CURRENT, sizeof(TOPIC_TELEM_CURRENT), "ngr/loco/
%s/telem/current", id);
    snprintf(TOPIC_TELEM_POWER, sizeof(TOPIC_TELEM_POWER), "ngr/loco/
%s/telem/power", id);
}

void mqttPublishThrottle() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_THROTTLE, String(rampTarget).c_str(), true);
}

void mqttPublishDirection() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_DIRECTION, String(direction).c_str(), true);
}

void mqttPublishBrake() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_BRAKE, String(brakeValue).c_str(), true);
}

void mqttPublishEstop() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_ESTOP, eStopEngaged ? "1" : "0", true);
}

void mqttPublishAuto() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_AUTO, dispatcherAuto ? "1" : "0", true);
}

void mqttPublishLowvolt() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_LOWVOLT, (voltageState == LOW_VOLTAGE) ? "1" : "0",
true);
}

```

```

}

void mqttPublishWarning(const char* msg) {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_WARNING, msg, false); // transient, not retained
    mqttWarningClearAtMs = (msg && msg[0] != '\0') ? millis() +
MQTT_WARNING_TTL_MS : 0;
}

void mqttPublishBlock() {
    if (!mqtt.connected()) return;
    mqtt.publish(TOPIC_STATE_BLOCK, blockName((BlockId)myCurrentBlock), true);
}

void mqttPublishTelemetry(float busVoltage) {
    if (!mqtt.connected()) return;
    char buf[16];
    dtostrf(busVoltage,          5, 2, buf); mqtt.publish(TOPIC_TELEM_VOLTAGE, buf,
true);
    dtostrf(current_mA / 1000.0f, 5, 3, buf); mqtt.publish(TOPIC_TELEM_CURRENT, buf,
true);
    dtostrf(power_mW / 1000.0f, 5, 3, buf); mqtt.publish(TOPIC_TELEM_POWER, buf,
true);
}

void mqttPublishFullStateSnapshot() {
    if (!mqtt.connected()) return;
    mqttPublishThrottle();
    mqttPublishDirection();
    mqttPublishBrake();
    mqttPublishEstop();
    mqttPublishAuto();
    mqttPublishLowvolt();
    mqttPublishBlock();
    mqttPublishTelemetry(lastBusVoltage);
}

//
=====
=====
// MANUAL DIRECTION / DISPATCHER AUTHORITY
//
=====
=====
void applyDirectionPin() {
    if (direction == DIRECTION_REVERSE) digitalWrite(MOTOR_DIR_PIN, LOW);
    if (direction == DIRECTION_FORWARD) digitalWrite(MOTOR_DIR_PIN, HIGH);
}

```



```

    // Neutral leaves the direction pin unchanged; PWM is forced to zero.
}

void enterDispatcherAutoMode() {
    // Authority transfer is allowed only from a safe, stationary,
    // forward-oriented manual state. Dispatcher GO starts motion later.
    if (eStopEngaged || voltageShutdownActive) {
        mqttPublishWarning("AUTO DENIED — CLEAR FAULT");
        mqttPublishAuto();
        return;
    }

    if (direction != DIRECTION_FORWARD) {
        mqttPublishWarning("AUTO DENIED — SELECT FORWARD");
        mqttPublishAuto();
        return;
    }

    if (rampCurrent > STOPPED_PWM_THRESHOLD || rampTarget != 0) {
        mqttPublishWarning("AUTO DENIED — STOP LOCOMOTIVE FIRST");
        mqttPublishAuto();
        return;
    }

    dispatcherAuto = true;
    running        = false;
    rampTarget     = 0;
    brakeValue     = 0;
    mqttPublishAuto();
    mqttPublishThrottle();
    mqttPublishBrake();
    mqttPublishWarning("");
    Serial.println("MQTT AUTO: Dispatcher authority accepted; awaiting Dispatcher GO.");
}

void leaveDispatcherAutoMode() {
    // The Dispatcher dashboard is the intended discharge path.
    // Return to manual without leaving live CTO/CE motion state behind.
    dispatcherAuto = false;
    running        = false;
    rampTarget     = 0;
    brakeValue     = 0;

    ceActive       = false;
    ceRole         = CE_NONE;
    ceExitArmed    = false;
    ctoOrderRole   = CE_NONE;
}

```

```

ceExpressStationCounter = 0;
clearCeTrailingCandidateState();

clearPeerTable();
resetEncounterState();
pendingStartActive = false;
pendingReleaseActive = false;

mqttPublishAuto();
mqttPublishThrottle();
mqttPublishBrake();
mqttPublishWarning("");
Serial.println("MQTT AUTO: Dispatcher authority released; manual control restored.");
}

//
=====
=====
// INA219 SERVICE — GoldCore v1.7 protection/telemetry adapted for combo sketch
//
=====
=====
void serviceINA219() {
    static unsigned long lastProtectMs = 0;
    static unsigned long lastTelemetryMs = 0;
    static unsigned long lastSerialMs = 0;

    const unsigned long now = millis();
    if (now - lastProtectMs < 250UL) return;
    lastProtectMs = now;

    float busVoltage = 0.0f;
    if (ina219Present) {
        busVoltage = ina219.getBusVoltage_V();
        current_mA = ina219.getCurrent_mA();
        power_mW = busVoltage * current_mA;
        lastBusVoltage = busVoltage;
    }

    if (busVoltage < DISCONNECTED_VOLTAGE_THRESHOLD) {
        voltageState = DISCONNECTED;
    } else if (busVoltage <= THROTTLE_LIMIT_VOLTAGE) {
        voltageState = LOW_VOLTAGE;
    } else {
        voltageState = NORMAL_OPERATION;
    }
}

```

```

if (voltageState == DISCONNECTED) {
    lowVoltageCounter      = 0;
    shutdownCounter        = 0;
    voltageShutdownActive   = false;
    lowVoltageThrottleActive = false;
    if (lastVoltageState != DISCONNECTED) mqttPublishLowvolt();
    lastVoltageState = voltageState;
    return;
}

if (busVoltage <= SHUTDOWN_VOLTAGE) {
    shutdownCounter++;
    if (shutdownCounter >= VOLTAGE_COUNTER_LIMIT && !voltageShutdownActive) {
        voltageShutdownActive = true;
        running               = false;
        rampTarget            = 0;
        rampCurrent           = 0;
        ledcWrite(MOTOR_PWM_PIN, 0);
        mqttPublishThrottle();
        mqttPublishWarning("LOW VOLTAGE SHUTDOWN");
        Serial.println("SHUTDOWN: voltage below threshold. Motor disabled.");
    }
} else {
    shutdownCounter = 0;
}

bool previousLimiter = lowVoltageThrottleActive;
if (!voltageShutdownActive && busVoltage <= THROTTLE_LIMIT_VOLTAGE) {
    lowVoltageCounter++;
    if (lowVoltageCounter >= VOLTAGE_COUNTER_LIMIT) lowVoltageThrottleActive =
true;
} else {
    lowVoltageCounter      = 0;
    lowVoltageThrottleActive = false;
}
if (previousLimiter != lowVoltageThrottleActive) mqttPublishLowvolt();

if (voltageShutdownActive && busVoltage >= RECOVERY_VOLTAGE && rampTarget
== 0) {
    voltageShutdownActive   = false;
    lowVoltageThrottleActive = false;
    mqttPublishLowvolt();
    mqttPublishWarning("");
    Serial.println("Voltage recovered. Motor control available again.");
}

if (now - lastTelemetryMs >= 2000UL) {

```

```

    lastTelemetryMs = now;
    mqttPublishTelemetry(busVoltage);
    mqttPublishLowvolt();
}

if (now - lastSerialMs >= 30000UL) {
    lastSerialMs = now;
    Serial.printf("INA I V=%.2fV I=%.1fmA P=%.0fmW I LOW=%d SD=%d\n",
        busVoltage, current_mA, power_mW,
        lowVoltageThrottleActive ? 1 : 0,
        voltageShutdownActive ? 1 : 0);
}

lastVoltageState = voltageState;
}

//
=====
=====
// BLOCK TOPOLOGY
//
=====
=====
BlockId previousBlock(BlockId b) {
    switch (b) {
        case BLOCK_PATIO: return BLOCK_GRILLERS;
        case BLOCK_BAMBOO: return BLOCK_PATIO;
        case BLOCK_ARCHES: return BLOCK_BAMBOO;
        case BLOCK_GRILLERS: return BLOCK_ARCHES;
        default: return BLOCK_UNKNOWN;
    }
}

BlockId nextBlock(BlockId b) {
    switch (b) {
        case BLOCK_PATIO: return BLOCK_BAMBOO;
        case BLOCK_BAMBOO: return BLOCK_ARCHES;
        case BLOCK_ARCHES: return BLOCK_GRILLERS;
        case BLOCK_GRILLERS: return BLOCK_PATIO;
        default: return BLOCK_UNKNOWN;
    }
}

// Distance in operating direction from follower/current block to leader block.
// 0 = same block, 1 = next block ahead, 2 = two blocks ahead, 3 = too far / relationship
broken.

```

```

int blocksAhead(uint8_t fromBlock, uint8_t toBlock) {
    if (fromBlock == BLOCK_UNKNOWN || toBlock == BLOCK_UNKNOWN) return 99;
    BlockId b = (BlockId)fromBlock;
    for (int d = 0; d <= 3; d++) {
        if ((uint8_t)b == toBlock) return d;
        b = nextBlock(b);
        if (b == BLOCK_UNKNOWN) return 99;
    }
    return 99;
}

```

```

bool leaderDepartureWithinTwoBlocks(uint8_t leaderBlock) {
    int d = blocksAhead(myCurrentBlock, leaderBlock);
    return d <= 2;
}

```

```

void breakTrailingRelationshipIfTooFar(uint8_t leaderBlock) {
    int d = blocksAhead(myCurrentBlock, leaderBlock);
    if (d > 2) {
        Serial.printf("TRAILING: leader departure too far away — breaking relationship.
myBlock=%s leaderBlock=%s distance=%d\n",
            blockName((BlockId)myCurrentBlock),
            blockName((BlockId)leaderBlock),
            d);
        resetEncounterState();
        stopReason = STOP_STATION_DWELL;
    }
}

```

```

const char* blockName(BlockId b) {
    switch (b) {
        case BLOCK_PATIO: return "PATIO";
        case BLOCK_BAMBOO: return "BAMBOO";
        case BLOCK_ARCHES: return "ARCHES";
        case BLOCK_GRILLERS: return "GRILLERS";
        default: return "UNKNOWN";
    }
}

```

```

const char* stopReasonName(StopReason r) {
    switch (r) {
        case STOP_STATION_DWELL: return "STATION_DWELL";
        case STOP_CTO_TRAILING: return "CTO_TRAILING";
        case STOP_CTO_LEADER_HOLD: return "CTO_LEADER_HOLD";
        case STOP_ESTOP: return "ESTOP";
        default: return "NONE";
    }
}

```

```
}
```

```
const char* trailingSubStateName(TrailingSubState s) {  
    switch (s) {  
        case TR_WAIT_LEADER_DEPART: return "WAIT_LEADER_DEPART";  
        case TR_FOLLOW_DELAY:        return "FOLLOW_DELAY";  
        case TR_ADVANCE_TO_PLATFORM: return "ADVANCE_TO_PLATFORM";  
        case TR_PLATFORM_STOP:       return "PLATFORM_STOP";  
        case TR_PLATFORM_DWELL:      return "PLATFORM_DWELL";  
        default:                     return "IDLE";  
    }  
}
```

```
const char* statusEventName(StatusEvent ev) {  
    switch (ev) {  
        case ST_BLOCK_DETECTED:      return "BLOCK_DETECTED";  
        case ST_ROLE_ASSIGNED:        return "ROLE_ASSIGNED";  
        case ST_RAMP_DELAY_START:     return "RAMP_DELAY_START";  
        case ST BRAKING:              return "BRAKING";  
        case ST_CONFIRMED_STOP:       return "CONFIRMED_STOP";  
        case ST_RELEASE_SENT:         return "RELEASE_SENT";  
        case ST_WAIT_LEADER:          return "WAIT_LEADER";  
        case ST_ADVANCE_START:        return "ADVANCE_START";  
        case ST_ADVANCE_END:          return "ADVANCE_END";  
        case ST_PLATFORM_STOP:        return "PLATFORM_STOP";  
        case ST_PLATFORM_DWELL:       return "PLATFORM_DWELL";  
        case ST_DEPARTING:            return "DEPARTING";  
        case ST_LEADER_WAITING:       return "LEADER_WAITING";  
        case ST_LEADER_RELEASED:      return "LEADER_RELEASED";  
        case ST_CE_LOCAL_EXIT_TO_CTO_LEADER: return  
"CE_LOCAL_EXIT_TO_CTO";  
        default:                     return "UNKNOWN";  
    }  
}
```

```
//
```

```
=====
```

```
// PEER TABLE MANAGEMENT
```

```
//
```

```
=====
```

```
int findPeer(uint32_t id) {  
    for (int i = 0; i < MAX_PEERS; i++)  
        if (peers[i].locold == id) return i;  
    return -1;  
}
```

```

int findEmptyPeerSlot() {
    for (int i = 0; i < MAX_PEERS; i++)
        if (peers[i].locold == 0) return i;
    return -1;
}

void updatePeer(uint32_t locold, uint8_t blockId, uint32_t seq) {
    int idx = findPeer(locold);
    if (idx >= 0) {
        peers[idx].lastBlock = blockId;
        peers[idx].lastSeq = seq;
        peers[idx].lastSeen = millis();
        updateCeExitArmedFromGeometry();
        return;
    }

    int slot = findEmptyPeerSlot();
    if (slot >= 0) {
        peers[slot] = { locold, blockId, seq, millis() };
        Serial.printf("New peer registered: %lu\n", (unsigned long)locold);
        updateCeExitArmedFromGeometry();
    } else {
        Serial.println("WARNING: peer table full");
    }
}

bool peerIsLogicFresh(const PeerState& p) {
    if (p.locold == 0) return false;
    return (millis() - p.lastSeen) <= LOGIC_FRESH_LIMIT_MS;
}

void checkPeerHealth() {
    for (int i = 0; i < MAX_PEERS; i++) {
        if (peers[i].locold == 0) continue;
        unsigned long age = millis() - peers[i].lastSeen;
        if (age > HEALTH_STOP_LIMIT_MS) Serial.printf("HEALTH STOP: peer %lu\n", (unsigned long)peers[i].locold);
        else if (age > HEALTH_WARN_LIMIT_MS) Serial.printf("HEALTH WARN: peer %lu\n", (unsigned long)peers[i].locold);
    }
}

void clearPeerTable() {
    for (int i = 0; i < MAX_PEERS; i++) peers[i] = {0, BLOCK_UNKNOWN, 0, 0};
    clearStickyLeaderDeparture();
}

```

```

//
=====
=====
// CTO ROLE EVALUATION
//
=====
=====
bool findTrailingEncounterPeer(uint8_t myBlock, uint32_t* out) {
    if (myBlock == BLOCK_UNKNOWN) return false;
    for (int i = 0; i < MAX_PEERS; i++) {
        if (peers[i].locold == 0) continue;
        if (!peerIsLogicFresh(peers[i])) continue;
        if (peers[i].lastBlock == myBlock) {
            *out = peers[i].locold;
            return true;
        }
    }
    return false;
}

bool findLeadingEncounterPeer(uint8_t myBlock, uint32_t* out) {
    if (myBlock == BLOCK_UNKNOWN) return false;
    BlockId prev = previousBlock((BlockId)myBlock);
    if (prev == BLOCK_UNKNOWN) return false;
    for (int i = 0; i < MAX_PEERS; i++) {
        if (peers[i].locold == 0) continue;
        if (!peerIsLogicFresh(peers[i])) continue;
        if (peers[i].lastBlock == (uint8_t)prev) {
            *out = peers[i].locold;
            return true;
        }
    }
    return false;
}

uint32_t makeEncounterId(uint8_t blockId) {
    localEncounterCounter++;
    return ((LOCO_ID & 0xFFFFUL) << 16) ^ ((uint32_t)blockId << 8) ^
(localEncounterCounter & 0xFFUL);
}

void resetEncounterState() {
    encounterRole      = ROLE_NONE;
    leaderHoldActive   = false;
    leaderReleaseReceived = false;
    leaderHoldStartedAt = 0;
}

```



```
holdingForTrailingId = 0;
holdingEncounterId   = 0;
holdingBlock         = BLOCK_UNKNOWN;
```

```
trailingEncounterActive = false;
trailingReleaseSent     = false;
trailingLeaderId        = 0;
trailingEncounterId     = 0;
trailingEncounterBlock  = BLOCK_UNKNOWN;
```

```
trailingSubState       = TR_IDLE;
departingReceived      = false;
leaderReachedNextBlock = false;
trailingTimerStart     = 0;
```

```
confirmedStopped      = false;
stoppedDebounceActive = false;
stoppedDebounceStart  = 0;
```

```
// Defensive reset. The CE Local -> CTO Leader path sets this again only
// after ordinary leader assignment is confirmed.
```

```
postCeLeaderFirstStopDelayActive = false;
```

```
}
```

```
void clearStickyLeaderDeparture() {
    stickyDepartingActive = false;
    stickyDepartingLeaderId = 0;
    stickyDepartingBlock    = BLOCK_UNKNOWN;
    stickyDepartingEncounterId = 0;
}
```

```
// Clear live CTO encounter behavior without clearing peer-table knowledge,
// run permission, or current motor output.
```

```
void clearCtoToFreshRunningState() {
    resetEncounterState();
    pendingStartActive = false;
    pendingReleaseActive = false;
    clearStickyLeaderDeparture();
}
```

```
stopReason          = STOP_NONE;
detectState          = DETECT_RUNNING;
rampDelayActive      = false;
rampDelayStart       = 0;
stateStart           = 0;
confirmedStopped     = false;
stoppedDebounceActive = false;
stoppedDebounceStart = 0;
```

```

}

void clearCeTrailingCandidateState() {
    ceTrailingCandidateActive = false;
    ceTrailingCandidateLeaderId = 0;
    ceTrailingCandidateBlock = BLOCK_UNKNOWN;
    ceTrailingCandidateTimerStart = 0;
}

void beginCeTrailingCandidate(uint32_t leaderId, uint8_t blockId) {
    ceTrailingCandidateActive = true;
    ceTrailingCandidateLeaderId = leaderId;
    ceTrailingCandidateBlock = blockId;
    ceTrailingCandidateTimerStart = 0;
}

void resumeUnpairedAfterCeTrailingCandidate(int resumePwm) {
    clearCeTrailingCandidateState();
    clearCtoToFreshRunningState();
    if (running) rampTarget = resumePwm;
}

bool getFreshPeerBlock(uint8_t* outBlock) {
    for (int i = 0; i < MAX_PEERS; i++) {
        if (peers[i].locold == 0) continue;
        if (!peerIsLogicFresh(peers[i])) continue;
        if (peers[i].lastBlock == BLOCK_UNKNOWN) continue;
        *outBlock = peers[i].lastBlock;
        return true;
    }
    return false;
}

void startCircuitExpressIfEligible() {
    if (ceActive) return;
    if (ctoOrderRole == CE_NONE) return;

    ceSavedCruisePwm = cruisePwm;
    ceRole = ctoOrderRole;
    ceActive = true;
    ceExitArmed = false;
    ceExpressStationCounter = 0;
    clearCeTrailingCandidateState();

    // CTO is suppressed during CE. Retain only the CE role chosen from current CTO
    order.
    ctoOrderRole = CE_NONE;

```

```

clearCtoToFreshRunningState();

if (ceRole == CE_EXPRESS) {
    cruisePwm = CE_EXPRESS_PWM;
    if (running) rampTarget = CE_EXPRESS_PWM;
} else {
    cruisePwm = CE_LOCAL_PWM;
    if (running) rampTarget = CE_LOCAL_PWM;
}
}

// Clear CE state and restore the pre-CE cruise PWM.
// CTO re-entry itself is intentionally handled by ordinary GoldCore Q1/Q2
// immediately after this function is called at a qualifying block entry.
void stopCircuitExpressStateOnly() {
    ceActive    = false;
    ceRole      = CE_NONE;
    ceExitArmed = false;
    ctoOrderRole = CE_NONE;
    cruisePwm   = ceSavedCruisePwm;
    ceExpressStationCounter = 0;
    clearCeTrailingCandidateState();
}

// CE exit remains suppressed until two-block separation has been proven.
void updateCeExitArmedFromGeometry() {
    if (!ceActive || ceExitArmed) return;
    if (myCurrentBlock == BLOCK_UNKNOWN) return;

    uint8_t peerBlock = BLOCK_UNKNOWN;
    if (!getFreshPeerBlock(&peerBlock)) return;

    int separation = 0;
    if (ceRole == CE_EXPRESS) {
        separation = blocksAhead(peerBlock, myCurrentBlock);
    } else if (ceRole == CE_LOCAL) {
        separation = blocksAhead(myCurrentBlock, peerBlock);
    }

    if (separation >= 2) ceExitArmed = true;
}

// Return-to-CTO structure requested for CE:
// Express same-block catch -> clear CE, ordinary Q1 makes it CTO trailing.
// Local entering a block with Express in previous block -> clear CE, ordinary Q2
// makes it CTO leader.
// No separate CE encounter ownership or CE release choreography is added here.

```

```

bool tryCeReturnToCtoAtBlock(BlockId b) {
    if (!ceActive || !ceExitArmed) return false;

    uint8_t peerBlock = BLOCK_UNKNOWN;
    if (!getFreshPeerBlock(&peerBlock)) return false;

    if (ceRole == CE_EXPRESS && peerBlock == (uint8_t)b) {
        stopCircuitExpressStateOnly();
        clearCtoToFreshRunningState();
        evaluateCtoAtBlock(b);
        if (trailingEncounterActive) {
            beginCeTrailingCandidate(trailingLeaderId, trailingEncounterBlock);
        }
        return true;
    }

    if (ceRole == CE_LOCAL && peerBlock == (uint8_t)previousBlock(b)) {
        stopCircuitExpressStateOnly();
        clearCtoToFreshRunningState();

        // Passive diagnostic preserved from v1.2:
        // capture exact post-CE, pre-CTO-leader state.
        sendCeLocalExitToCtoLeaderStatus();

        evaluateCtoAtBlock(b);

        // v1.3 spacing fix:
        // If ordinary CTO evaluation confirms that this former Local is now
        // the CTO leader, use the longer CE Local ramp-delay distance for this
        // FIRST post-CE leader stop only.
        if (leaderHoldActive && stopReason == STOP_CTO_LEADER_HOLD) {
            postCeLeaderFirstStopDelayActive = true;
        }

        return true;
    }

    return false;
}

void applyStickyLeaderDepartureIfMatching() {
    if (trailingEncounterActive &&
        !departingReceived &&
        stickyDepartingActive &&
        stickyDepartingLeaderId == trailingLeaderId) {

        if (!leaderDepartureWithinTwoBlocks(stickyDepartingBlock)) {

```

```

        breakTrailingRelationshipIfTooFar(stickyDepartingBlock);
        clearStickyLeaderDeparture();
        return;
    }

    departingReceived = true;
    Serial.printf("TRAILING: sticky DEPARTING accepted from leader=%lu block=%s
encounter=%lu\n",
        (unsigned long)stickyDepartingLeaderId,
        blockName((BlockId)stickyDepartingBlock),
        (unsigned long)stickyDepartingEncounterId);
    clearStickyLeaderDeparture();
}
}

//
=====
=====
// applyPendingEncounterIfMatching
//
=====
=====
void applyPendingEncounterIfMatching(uint32_t trailingId, uint8_t blockId) {
    // ENCOUNTER_START — block-specific: accept only if block matches
    if (pendingStartActive &&
        pendingStartTrailingId == trailingId &&
        pendingStartBlock == blockId) {
        holdingEncounterId = pendingStartEncounterId;
        pendingStartActive = false;
    }

    // RELEASE — block-independent: trailing loco stopped, we may depart.
    if (pendingReleaseActive &&
        pendingReleaseTrailingId == trailingId &&
        (holdingEncounterId == 0 || holdingEncounterId == pendingReleaseEncounterId)) {
        holdingEncounterId = pendingReleaseEncounterId;
        leaderReleaseReceived = true;
        pendingReleaseActive = false;
    }
}

void evaluateCtoAtBlock(BlockId b) {
    resetEncounterState();
    checkPeerHealth();
    uint32_t peerId = 0;

    if (findTrailingEncounterPeer((uint8_t)b, &peerId)) {

```

```

    encounterRole      = ROLE_TRAILING;
    ctoOrderRole       = CE_LOCAL;
    trailingEncounterActive = true;
    trailingReleaseSent  = false;
    trailingLeaderId     = peerId;
    trailingEncounterBlock = (uint8_t)b;
    trailingEncounterId  = makeEncounterId((uint8_t)b);
    stopReason           = STOP_CTO_TRAILING;
    trailingSubState      = TR_WAIT_LEADER_DEPART;
    Serial.printf("CTO Q1: TRAILING. leader=%lu block=%s encounter=%lu\n",
        (unsigned long)trailingLeaderId, blockName(b),
        (unsigned long)trailingEncounterId);
    applyStickyLeaderDepartureIfMatching();
    return;
}

```

```

if (findLeadingEncounterPeer((uint8_t)b, &peerId)) {
    encounterRole      = ROLE_LEADING;
    ctoOrderRole       = CE_EXPRESS;
    leaderHoldActive    = true;
    leaderReleaseReceived = false;
    holdingForTrailingId = peerId;
    holdingEncounterId  = 0;
    holdingBlock         = (uint8_t)b;
    stopReason           = STOP_CTO_LEADER_HOLD;
    pendingReleaseActive = false;
    leaderHoldStartedAt  = millis();
    Serial.printf("CTO Q2: LEADING. trailing=%lu block=%s\n",
        (unsigned long)holdingForTrailingId, blockName(b));
    return;
}

```

```

    encounterRole = ROLE_NONE;
    ctoOrderRole  = CE_NONE;
    stopReason    = STOP_STATION_DWELL;
}

```

```
//
```

```
=====
```

```
=====
```

```
// HALL SENSOR FUNCTIONS
```

```
//
```

```
=====
```

```
=====
```

```

int readAveragedADC() {
    long total = 0;
    for (int i = 0; i < ADC_SAMPLES; i++) {

```

```

    total += analogRead(HALL_PIN);
    delayMicroseconds(300);
}
return (int)(total / ADC_SAMPLES);
}

```

```

HallState classifyHall(int raw, HallState prev) {
    switch (prev) {
        case HALL_NORTH:
            if (raw < northExitThreshold) return (raw <= southEnterThreshold) ? HALL_SOUTH :
HALL_NONE;
            return HALL_NORTH;

        case HALL_SOUTH:
            if (raw > southExitThreshold) return (raw >= northEnterThreshold) ? HALL_NORTH :
HALL_NONE;
            return HALL_SOUTH;

        default:
            if (raw >= northEnterThreshold) return HALL_NORTH;
            if (raw <= southEnterThreshold) return HALL_SOUTH;
            return HALL_NONE;
    }
}

```

```

HallState readHall() {
    return classifyHall(readAveragedADC(), currentHall);
}

```

```

void calibrateBaseline() {
    unsigned long start = millis();
    long total = 0;
    int count = 0;

    while (millis() - start < CALIBRATION_TIME_MS) {
        total += readAveragedADC();
        count++;
        delay(20);
    }
}

```

```

baselineCounts    = (count > 0) ? (int)(total / count) : readAveragedADC();
northEnterThreshold = baselineCounts + DEFAULT_DEADBAND_COUNTS +
TRIGGER_MARGIN_COUNTS;
northExitThreshold = baselineCounts + DEFAULT_DEADBAND_COUNTS;
southEnterThreshold = baselineCounts - DEFAULT_DEADBAND_COUNTS -
TRIGGER_MARGIN_COUNTS;
southExitThreshold = baselineCounts - DEFAULT_DEADBAND_COUNTS;

```

```
}
```

```
BlockId decodeBlock(HallState a, HallState b) {  
    if (a == HALL_SOUTH && b == HALL_SOUTH) return BLOCK_PATIO;  
    if (a == HALL_SOUTH && b == HALL_NORTH) return BLOCK_BAMBOO;  
    if (a == HALL_NORTH && b == HALL_NORTH) return BLOCK_ARCHES;  
    if (a == HALL_NORTH && b == HALL_SOUTH) return BLOCK_GRILLERS;  
    return BLOCK_UNKNOWN;  
}
```

```
//
```

```
=====
```

```
// RADIO — SEND FUNCTIONS
```

```
//
```

```
=====
```

```
void sendBlockReport(BlockId b) {  
    TrainPacket pkt = {};  
    pkt.magic = MAGIC;  
    pkt.version = VERSION;  
    pkt.msgType = MSG_TYPE_BLOCK;  
    pkt.senderId = LOCO_ID;  
    pkt.locId = LOCO_ID;  
    pkt.blockId = (uint8_t)b;  
    pkt.sequenceNumber = ++txSequence;  
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));  
}
```

```
void sendEncounterStart(uint32_t leaderId, uint8_t blockId, uint32_t encounterId) {  
    TrainPacket pkt = {};  
    pkt.magic = MAGIC;  
    pkt.version = VERSION;  
    pkt.msgType = MSG_TYPE_ENCOUNTER_START;  
    pkt.senderId = LOCO_ID;  
    pkt.locId = leaderId;  
    pkt.blockId = blockId;  
    pkt.sequenceNumber = ++txSequence;  
    pkt.encounterId = encounterId;  
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));  
}
```

```
void sendRelease(uint32_t leaderId, uint8_t blockId, uint32_t encounterId) {  
    TrainPacket pkt = {};  
    pkt.magic = MAGIC;  
    pkt.version = VERSION;  
    pkt.msgType = MSG_TYPE_RELEASE;
```



```

    pkt.senderId = LOCO_ID;
    pkt.locold = leaderId;
    pkt.blockId = blockId;
    pkt.sequenceNumber = ++txSequence;
    pkt.encounterId = encounterId;
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));
}

```

```

void sendDeparting(uint32_t targetId, uint8_t blockId, uint32_t encounterId) {
    TrainPacket pkt = {};
    pkt.magic = MAGIC;
    pkt.version = VERSION;
    pkt.msgType = MSG_TYPE_DEPARTING;
    pkt.senderId = LOCO_ID;
    pkt.locold = targetId;
    pkt.blockId = blockId;
    pkt.sequenceNumber = ++txSequence;
    pkt.encounterId = encounterId;
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));
}

```

```

void sendEstopAck() {
    TrainPacket pkt = {};
    pkt.magic = MAGIC;
    pkt.version = VERSION;
    pkt.msgType = MSG_TYPE_ESTOP;
    pkt.senderId = LOCO_ID;
    pkt.locold = LOCO_ID;
    pkt.flags = 1;
    pkt.sequenceNumber = ++txSequence;
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));
}

```

```
//
```

```
=====
```

```
=====
```

```
// sendStatus
```

```
//
```

```
=====
```

```
=====
```

```

void sendStatus(StatusEvent ev) {
    TrainPacket pkt = {};
    pkt.magic = MAGIC;
    pkt.version = VERSION;
    pkt.msgType = MSG_TYPE_STATUS;
    pkt.senderId = LOCO_ID;
    pkt.locold = LOCO_ID;
}

```

```

    pkt.blockId = myCurrentBlock;
    pkt.cmd     = (uint8_t)ev;
    pkt.value   = (uint16_t)rampCurrent;
    pkt.flags   = (confirmedStopped ? 0x01 : 0x00)
        | (departingReceived ? 0x02 : 0x00);
    pkt.encounterId = trailingEncounterActive ? trailingEncounterId : holdingEncounterId;
    pkt.sequenceNumber = ++txSequence;
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));
}

```

```
//
```

```
=====
=====
```

```
// CE Local exit diagnostic — reporting only
```

```
//
```

```
// Emitted only when the former CE Local qualifies to leave CE and re-enter
```

```
// ordinary CTO as the new leader. This does not alter control logic.
```

```
//
```

```
=====
=====
```

```
void sendCeLocalExitToCtoLeaderStatus() {
```

```
    TrainPacket pkt = {};
```

```
    pkt.magic   = MAGIC;
```

```
    pkt.version = VERSION;
```

```
    pkt.msgType = MSG_TYPE_STATUS;
```

```
    pkt.senderId = LOCO_ID;
```

```
    pkt.locId   = LOCO_ID;
```

```
    pkt.blockId = myCurrentBlock;
```

```
    pkt.cmd     = ST_CE_LOCAL_EXIT_TO_CTO_LEADER;
```

```
    pkt.value   = (uint16_t)rampCurrent;
```

```
    pkt.roleId  = (uint8_t)constrain(cruisePwm, 0, 255);
```

```
    pkt.paramId = (uint8_t)constrain(rampTarget, 0, 255);
```

```
    pkt.flags   = (ceActive ? 0x01 : 0x00)
```

```
        | (((uint8_t)ceRole & 0x03) << 1);
```

```
    pkt.sequenceNumber = ++txSequence;
```

```
    esp_now_send(BROADCAST_MAC, (uint8_t*)&pkt, sizeof(pkt));
```

```
}
```

```
//
```

```
=====
=====
```

```
// MQTT COMMAND HANDLER
```

```
// Matches the Flask app currently running on ngr-pi.
```

```

//
=====
=====
void onMQTTMessage(char* topic, byte* payload, unsigned int length) {
    String t(topic);
    String msg;
    for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
    int val = msg.toInt();

    Serial.printf("MQTT RX [%s] = %s\n", t.c_str(), msg.c_str());

    // -----
    // Manual throttle
    // -----
    if (t == TOPIC_CMD_THROTTLE) {
        if (dispatcherAuto) {
            mqttPublishWarning("DISPATCHER IN CONTROL");
            mqttPublishThrottle();
            return;
        }
        if (eStopEngaged || voltageShutdownActive) {
            mqttPublishThrottle();
            return;
        }
        if (direction == DIRECTION_NEUTRAL) {
            mqttPublishWarning("SELECT FORWARD OR REVERSE");
            mqttPublishThrottle();
            return;
        }
        rampTarget = constrain(val, 0, 255);
        mqttPublishThrottle();
        return;
    }

    // -----
    // Manual direction
    // -----
    if (t == TOPIC_CMD_DIRECTION) {
        if (dispatcherAuto) {
            mqttPublishWarning("DISPATCHER IN CONTROL");
            mqttPublishDirection();
            return;
        }

        if (rampCurrent > STOPPED_PWM_THRESHOLD) {
            mqttPublishWarning("WAIT FOR COMPLETE STOP");
        }
    }
}

```

```

    mqttPublishDirection();
    return;
}

if (rampCurrent != 0 || rampTarget != 0) {
    rampCurrent = 0;
    rampTarget = 0;
    ledcWrite(MOTOR_PWM_PIN, 0);
    mqttPublishThrottle();
}

direction = constrain(val, DIRECTION_REVERSE, DIRECTION_FORWARD);
applyDirectionPin();
mqttPublishDirection();
return;
}

// -----
// Manual brake
// -----
if (t == TOPIC_CMD_BRAKE) {
    if (dispatcherAuto) {
        mqttPublishWarning("DISPATCHER IN CONTROL");
        mqttPublishBrake();
        return;
    }

    brakeValue = constrain(val, 0, 255);
    mqttPublishBrake();
    return;
}

// -----
// Loco-page E-STOP toggle
// -----
if (t == TOPIC_CMD_ESTOP) {
    if (val == 1) {
        eStopEngaged = true;
        running = false;
        rampTarget = 0;
        rampCurrent = 0;
        ledcWrite(MOTOR_PWM_PIN, 0);
        stopReason = STOP_ESTOP;
        mqttPublishEstop();
        mqttPublishThrottle();
        mqttPublishWarning("E-STOP ACTIVE");
        Serial.println("MQTT E-STOP ON");
    }
}

```

```

    } else {
        eStopEngaged = false;
        if (stopReason == STOP_ESTOP) stopReason = STOP_NONE;
        mqttPublishEstop();
        mqttPublishWarning("");
        Serial.println("MQTT E-STOP OFF");
    }
    return;
}

// -----
// Locomotive dashboard authority transfer
// auto=1: enroll under Dispatcher authority, but DO NOT start motion.
// auto=0: intentionally rejected. Discharge is Dispatcher-only and must
// arrive on cmd/dispatcher_release.
// -----
if (t == TOPIC_CMD_AUTO) {
    if (val == 1) {
        enterDispatcherAutoMode();
    } else {
        mqttPublishWarning("DISCHARGE REJECTED — USE DISPATCHER");
        mqttPublishAuto();
        Serial.println("MQTT unauthorized auto=0 discharge ignored");
    }
    return;
}

// -----
// Dispatcher-authorized discharge back to manual control.
// Flask END CTO publishes dispatcher_release=1 to both locomotives.
// -----
if (t == TOPIC_CMD_DISPATCHER_RELEASE) {
    if (val == 1) {
        leaveDispatcherAutoMode();
    }
    return;
}
}

//
=====
=====
// MQTT RECONNECT — non-blocking
//
=====
=====
void serviceMQTTReconnect() {

```

```

static unsigned long lastAttemptMs = 0;
if (mqtt.connected()) return;

unsigned long now = millis();
if (now - lastAttemptMs < 5000UL) return;
lastAttemptMs = now;

Serial.printf("MQTT connecting to %s:%d ...\n", MQTT_BROKER, MQTT_PORT);
if (mqtt.connect(MQTT_CLIENT_ID, NULL, NULL, TOPIC_ONLINE, 0, true, "0")) {
    mqtt.publish(TOPIC_ONLINE, "1", true);
    mqtt.subscribe(TOPIC_CMD_THROTTLE);
    mqtt.subscribe(TOPIC_CMD_DIRECTION);
    mqtt.subscribe(TOPIC_CMD_BRAKE);
    mqtt.subscribe(TOPIC_CMD_ESTOP);
    mqtt.subscribe(TOPIC_CMD_AUTO);
    mqtt.subscribe(TOPIC_CMD_DISPATCHER_RELEASE);
    Serial.println("MQTT connected. Loco command topics subscribed.");
    mqttPublishFullStateSnapshot();
} else {
    Serial.printf("MQTT connect failed rc=%d\n", mqtt.state());
}
}

//
=====
=====
// RADIO — RECEIVE HANDLER
//
=====
=====
void onReceive(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    if (len != (int)sizeof(TrainPacket)) return;

    TrainPacket pkt;
    memcpy(&pkt, data, sizeof(pkt));

    if (pkt.magic != MAGIC || pkt.version != VERSION) return;
    if (pkt.senderId == LOCO_ID) return;

    switch (pkt.msgType) {

    case MSG_TYPE_BLOCK:
        if (!dispatcherAuto) break;
        updatePeer(pkt.locold, pkt.blockId, pkt.sequenceNumber);

        if (stickyDepartingActive && stickyDepartingLeaderId == pkt.locold) {
            clearStickyLeaderDeparture();

```

```

}

if (trailingEncounterActive &&
    trailingLeaderId == pkt.locold &&
    departingReceived &&
    pkt.blockId != trailingEncounterBlock) {
    leaderReachedNextBlock = true;
    if (trailingSubState == TR_PLATFORM_DWELL && trailingTimerStart == 0) {
        trailingTimerStart = millis();
    }
}

// CE failed-recapture escape:
// If the former Local reports a later block before the temporary
// trailing-candidate timer expires, pairing did not complete here.
if (ceTrailingCandidateActive &&
    pkt.locold == ceTrailingCandidateLeaderId &&
    pkt.blockId != ceTrailingCandidateBlock) {
    resumeUnpairedAfterCeTrailingCandidate(cruisePwm);
}
break;

case MSG_TYPE_ENCOUNTER_START:
    if (!dispatcherAuto) break;
    if (ceActive) break;
    if (pkt.locold != LOCO_ID) return;

    pendingStartActive = true;
    pendingStartTrailingId = pkt.senderId;
    pendingStartEncounterId = pkt.encounterId;
    pendingStartBlock = pkt.blockId;

    if (leaderHoldActive &&
        holdingForTrailingId == pkt.senderId &&
        holdingBlock == pkt.blockId) {
        holdingEncounterId = pkt.encounterId;
        pendingStartActive = false;
    }
    break;

case MSG_TYPE_RELEASE:
    if (!dispatcherAuto) break;
    if (ceActive) break;
    if (pkt.locold != LOCO_ID) return;

    if (leaderHoldActive &&
        holdingForTrailingId == pkt.senderId &&

```

```

        (holdingEncounterId == 0 || holdingEncounterId == pkt.encounterId) &&
        millis() >= leaderHoldStartedAt) {
    holdingEncounterId = pkt.encounterId;
    leaderReleaseReceived = true;
    stateStart = millis();
    Serial.println("LEADER: RELEASE accepted — timestamp valid, gate unlocked");
} else {
    pendingReleaseActive = true;
    pendingReleaseTrailingId = pkt.senderId;
    pendingReleaseEncounterId = pkt.encounterId;
    pendingReleaseBlock = pkt.blockId;
}
break;

case MSG_TYPE_DEPARTING:
    if (!dispatcherAuto) break;
    if (ceActive) break;

    stickyDepartingActive = true;
    stickyDepartingLeaderId = pkt.senderId;
    stickyDepartingBlock = pkt.blockId;
    stickyDepartingEncounterId = pkt.encounterId;

    if (trailingEncounterActive &&
        trailingLeaderId == pkt.senderId &&
        (pkt.locold == 0 || pkt.locold == LOCO_ID)) {

        if (!leaderDepartureWithinTwoBlocks(pkt.blockId)) {
            breakTrailingRelationshipIfTooFar(pkt.blockId);
            clearStickyLeaderDeparture();
            break;
        }

        departingReceived = true;

        if (ceTrailingCandidateActive &&
            ceTrailingCandidateLeaderId == pkt.senderId) {
            clearCeTrailingCandidateState();
        }

        clearStickyLeaderDeparture();
    }
    break;

case MSG_TYPE_ESTOP:
    eStopEngaged = true;
    running = false;

```



```

rampTarget = 0;
rampCurrent = 0;
ledcWrite(MOTOR_PWM_PIN, 0);
stopReason = STOP_ESTOP;
ceActive = false;
ceRole = CE_NONE;
ceExitArmed = false;
ctoOrderRole = CE_NONE;
ceExpressStationCounter = 0;
clearCeTrailingCandidateState();
resetEncounterState();
sendEstopAck();
mqttPublishEstop();
mqttPublishThrottle();
mqttPublishWarning("E-STOP ACTIVE");
Serial.println("E-STOP received");
break;

```

```

case MSG_TYPE_STATUS:

```

```

    // Locos ignore STATUS packets from peers — Dispatcher only.
    break;

```

```

case MSG_TYPE_MANUAL:

```

```

    if (pkt.locold != 0 && pkt.locold != LOCO_ID) return;

```

```

    if (pkt.cmd == CMD_START) {

```

```

        if (!dispatcherAuto) {

```

```

            Serial.println("CMD: START ignored — loco is in MANUAL authority.");

```

```

            return;

```

```

        }

```

```

        if (stopReason != STOP_ESTOP && !stopEngaged && !voltageShutdownActive) {

```

```

            running = true;

```

```

            if (detectState == DETECT_RUNNING) rampTarget = cruisePwm;

```

```

            mqttPublishThrottle();

```

```

            Serial.println("CMD: START");

```

```

        }

```

```

    }

```

```

    else if (pkt.cmd == CMD_STOP) {

```

```

        running = false;

```

```

        rampTarget = 0;

```

```

        mqttPublishThrottle();

```

```

        Serial.println("CMD: STOP");

```

```

    }

```

```

    else if (pkt.cmd == CMD_SET_PWM) {

```

```

        if (!dispatcherAuto) {

```

```

            Serial.println("CMD: SET PWM ignored — loco is in MANUAL authority.");

```

```

            return;

```

```

    }
    cruisePwm = constrain((int)pkt.value, 0, 255);
    if (running && detectState == DETECT_RUNNING) rampTarget = cruisePwm;
    mqttPublishThrottle();
    Serial.printf("CMD: SET PWM = %d\n", cruisePwm);
}
else if (pkt.cmd == CMD_SET_ROLE_PARAM) {
    if (pkt.paramId == PARAM_RAMP_RATE) {
        setSharedRampRate((unsigned long)pkt.value);
    }
    else if (pkt.paramId == PARAM_RAMP_UP_RATE) {
        rampUpRateMs = (unsigned long)pkt.value;
        Serial.printf("CMD: rampUpRateMs = %lu ms\n", rampUpRateMs);
    }
    else if (pkt.roleId == ROLE_ID_TRAILING && pkt.paramId ==
PARAM_FOLLOW_DELAY) {
        trailingParams.followDelayMs = (unsigned long)pkt.value;
        Serial.printf("CMD: TRAILING followDelayMs = %lu\n",
trailingParams.followDelayMs);
    }
    else if (pkt.roleId == ROLE_ID_TRAILING && pkt.paramId ==
PARAM_ADVANCE_MS) {
        trailingParams.advanceMs = (unsigned long)pkt.value;
        Serial.printf("CMD: TRAILING advanceMs = %lu\n", trailingParams.advanceMs);
    }
    else if (pkt.roleId == ROLE_ID_TRAILING && pkt.paramId ==
PARAM_PLATFORM_DWELL) {
        trailingParams.platformDwellMs = (unsigned long)pkt.value;
        Serial.printf("CMD: TRAILING platformDwellMs = %lu\n",
trailingParams.platformDwellMs);
    }
    else if (pkt.roleId == ROLE_ID_TRAILING && pkt.paramId ==
PARAM_ADVANCE_PWM) {
        trailingAdvancePwmOverride = constrain((int)pkt.value, 0, 255);
        Serial.printf("CMD: TRAILING advancePwmOverride = %d\n",
trailingAdvancePwmOverride);
    }
    else {
        RoleProfile& p = profileForRole(pkt.roleId);
        if (pkt.paramId == PARAM_RAMP_DELAY) {
            p.rampDelayMs = (unsigned long)pkt.value;
            Serial.printf("CMD: ROLE %u rampDelayMs=%lu\n", pkt.roleId,
p.rampDelayMs);
        }
        else if (pkt.paramId == PARAM_DWELL) {
            p.dwellMs = (unsigned long)pkt.value;
            Serial.printf("CMD: ROLE %u dwellMs=%lu\n", pkt.roleId, p.dwellMs);
        }
    }
}

```

```

    }
  }
}
else if (pkt.cmd == CMD_CLEAR_ALL) {
  clearPeerTable();
  resetEncounterState();
  ceActive = false;
  ceRole = CE_NONE;
  ceExitArmed = false;
  ctoOrderRole = CE_NONE;
  ceExpressStationCounter = 0;
  clearCeTrailingCandidateState();
  pendingStartActive = false;
  pendingReleaseActive = false;
  Serial.println("CMD: CLEAR ALL");
}
else if (pkt.cmd == CMD_CE_START) {
  if (!dispatcherAuto) {
    Serial.println("CMD: CE ignored — loco is in MANUAL authority.");
    return;
  }
  startCircuitExpressIfEligible();
  mqttPublishThrottle();
}
break;
}
}

//
=====
=====
// RAMP SERVICE
//
=====
=====
void serviceRamp() {
  if (eStopEngaged || direction == DIRECTION_NEUTRAL || voltageShutdownActive) {
    if (rampCurrent != 0) {
      rampCurrent = 0;
      ledcWrite(MOTOR_PWM_PIN, 0);
    }
  }
  return;
}

int effectiveTarget = rampTarget;
if (lowVoltageThrottleActive && effectiveTarget > MIN_THROTTLE_PWM) {
  effectiveTarget = MIN_THROTTLE_PWM;
}

```

```

}

if (rampCurrent == effectiveTarget) return;

unsigned long delayMs;
if (rampCurrent > effectiveTarget) {
  if (dispatcherAuto) {
    delayMs = activeProfile().rampRateMs;
  } else {
    // GoldCore manual brake slider: 0 = long coast, 255 = strong brake.
    delayMs = (unsigned long)map((brakeValue * brakeValue) / 255, 0, 255, 300, 1);
  }
} else {
  delayMs = dispatcherAuto ? rampUpRateMs : RAMP_UP_DELAY_MS;
}

if (millis() - lastRamp < delayMs) return;

lastRamp = millis();
rampCurrent += (rampCurrent < effectiveTarget) ? 1 : -1;
ledcWrite(MOTOR_PWM_PIN, rampCurrent);
}

//
=====
=====
// CONFIRMED-STOP DEBOUNCE
//
=====
=====
bool serviceStopDebounce() {
  if (confirmedStopped) return true;

  if (rampCurrent <= STOPPED_PWM_THRESHOLD) {
    if (!stoppedDebounceActive) {
      stoppedDebounceActive = true;
      stoppedDebounceStart = millis();
    }
    else if (millis() - stoppedDebounceStart >= STOPPED_DEBOUNCE_MS) {
      confirmedStopped = true;
      Serial.println("CONFIRMED STOP");
      sendStatus(ST_CONFIRMED_STOP);
      return true;
    }
  }
  else {
    stoppedDebounceActive = false;
  }
}

```

```

    }

    return false;
}

//
=====
=====
// TRAILING SUB-STATE MACHINE
//
=====
=====
void serviceTrailingSubState() {
    switch (trailingSubState) {

        case TR_WAIT_LEADER_DEPART:
            if (!departingReceived) {
                static unsigned long lastPrint = 0;
                if (millis() - lastPrint > 3000UL) {
                    lastPrint = millis();
                    sendStatus(ST_WAIT_LEADER);
                }
                return;
            }

            trailingSubState = TR_FOLLOW_DELAY;
            trailingTimerStart = millis();
            confirmedStopped = false;
            stoppedDebounceActive = false;
            break;

        case TR_FOLLOW_DELAY:
            if (millis() - trailingTimerStart < trailingParams.followDelayMs) return;

            trailingSubState = TR_ADVANCE_TO_PLATFORM;
            trailingTimerStart = millis();

            {
                int advancePwm = (trailingAdvancePwmOverride > 0)
                    ? trailingAdvancePwmOverride
                    : (cruisePwm / 2);
                rampTarget = advancePwm;
            }

            sendStatus(ST_ADVANCE_START);
            break;
    }
}

```

```

case TR_ADVANCE_TO_PLATFORM:
    if (millis() - trailingTimerStart < trailingParams.advanceMs) return;

    trailingSubState = TR_PLATFORM_STOP;
    rampTarget      = 0;
    sendStatus(ST_ADVANCE_END);
    break;

case TR_PLATFORM_STOP:
    if (!serviceStopDebounce()) return;

    trailingSubState = TR_PLATFORM_DWELL;
    trailingTimerStart = 0;
    sendStatus(ST_PLATFORM_DWELL);
    break;

case TR_PLATFORM_DWELL:
    if (!leaderReachedNextBlock) {
        return;
    }

    if (trailingTimerStart == 0) trailingTimerStart = millis();
    if (millis() - trailingTimerStart < trailingParams.platformDwellMs) return;

    Serial.println("TRAILING: leader reached next block — platform release delay
complete");
    detectState  = DETECT_RUNNING;
    stopReason   = STOP_NONE;
    myCurrentBlock = BLOCK_UNKNOWN;
    mqttPublishBlock();
    resetEncounterState();
    sendDeparting(0, BLOCK_UNKNOWN, 0);
    sendStatus(ST_DEPARTING);
    if (running) rampTarget = cruisePwm;
    break;

case TR_IDLE:
default:
    break;
}
}

//
=====
=====
// startDwellAtBlock
//

```

```

=====
=====
void startDwellAtBlock(BlockId b) {
    myCurrentBlock = (uint8_t)b;
    mqttPublishBlock();

    Serial.printf("BLOCK: %s\n", blockName(b));
    sendBlockReport(b);
    sendStatus(ST_BLOCK_DETECTED);

    lastBlockTime = millis();

    if (ceActive) {
        updateCeExitArmedFromGeometry();

        bool returnedToCto = tryCeReturnToCtoAtBlock(b);
        if (returnedToCto) {
            sendStatus(ST_ROLE_ASSIGNED);
            rampDelayActive = true;
            rampDelayStart = millis();
            stateStart = 0;
            detectState = DETECT_DWELLING;
            sendStatus(ST_RAMP_DELAY_START);
            return;
        }

        if (ceRole == CE_EXPRESS) {
            ceExpressStationCounter++;

            // Limited-stop Express: skip two station detections, stop at the third.
            if (ceExpressStationCounter >= CE_EXPRESS_STOP_EVERY_N_STATIONS) {
                ceExpressStationCounter = 0;

                stopReason = STOP_STATION_DWELL;
                rampDelayActive = true;
                rampDelayStart = millis();
                stateStart = 0;
                detectState = DETECT_DWELLING;
                sendStatus(ST_RAMP_DELAY_START);
                return;
            }

            // Express bypass station.
            stopReason = STOP_NONE;
            detectState = DETECT_RUNNING;
            rampDelayActive = false;
            stateStart = 0;

```

```

    if (running) rampTarget = CE_EXPRESS_PWM;
    return;
}

// CE Local: ordinary brief station service, independent of CTO choreography.
stopReason    = STOP_STATION_DWELL;
rampDelayActive = true;
rampDelayStart = millis();
stateStart    = 0;
detectState    = DETECT_DWELLING;
sendStatus(ST_RAMP_DELAY_START);
return;
}

evaluateCtoAtBlock(b);
sendStatus(ST_ROLE_ASSIGNED);

rampDelayActive = true;
rampDelayStart = millis();
stateStart    = 0;
detectState    = DETECT_DWELLING;

sendStatus(ST_RAMP_DELAY_START);
}

//
=====
=====
// serviceDwelling
//
=====
=====
void serviceDwelling() {

    // BEACON — rebroadcast BLOCK every 3 seconds while dwelling.
    static unsigned long lastBeacon = 0;
    if (millis() - lastBeacon > 3000UL) {
        sendBlockReport((BlockId)myCurrentBlock);
        lastBeacon = millis();
    }

    // Phase 1 — ramp delay
    if (rampDelayActive) {
        if (millis() - rampDelayStart < activeRampDelayMs()) return;

        rampDelayActive = false;

```



```

// v1.3: the post-CE leader spacing override is consumed exactly when
// the first braking phase begins.
if (postCeLeaderFirstStopDelayActive &&
    stopReason == STOP_CTO_LEADER_HOLD) {
    postCeLeaderFirstStopDelayActive = false;
}

rampTarget = 0;
sendStatus(ST_BRAKING);
}

// Phase 2 — confirmed stop
if (!serviceStopDebounce()) return;

// Phase 3 — TRAILING
if (!ceActive && trailingEncounterActive) {
    if (!trailingReleaseSent) {

        sendEncounterStart(trailingLeaderId, trailingEncounterBlock, trailingEncounterId);

        delay(50);
        sendRelease(trailingLeaderId, trailingEncounterBlock, trailingEncounterId);

        delay(150);
        sendRelease(trailingLeaderId, trailingEncounterBlock, trailingEncounterId);

        delay(150);
        sendRelease(trailingLeaderId, trailingEncounterBlock, trailingEncounterId);

        trailingReleaseSent = true;

        if (ceTrailingCandidateActive && ceTrailingCandidateTimerStart == 0) {
            ceTrailingCandidateTimerStart = millis();
        }

        Serial.println("TRAILING: confirmed stop — ENCOUNTER_START + RELEASE
sent 3x");
        sendStatus(ST_RELEASE_SENT);
    }

    if (ceTrailingCandidateActive) {
        if (ceTrailingCandidateTimerStart != 0 &&
            millis() - ceTrailingCandidateTimerStart >=
CE_TRAILING_CANDIDATE_ESCAPE_MS) {

resumeUnpairedAfterCeTrailingCandidate(CE_TRAILING_CANDIDATE_CAUTION_PW
M);

```

```

    }
    return;
}

serviceTrailingSubState();
return;
}

// Phase 4 / 5 — dwell and LEADER gate
if (!ceActive && leaderHoldActive) {
    if (!leaderReleaseReceived) {
        static unsigned long lastPrint = 0;
        if (millis() - lastPrint > 3000UL) {
            lastPrint = millis();
            sendStatus(ST_LEADER_WAITING);
        }
        return;
    }

    if (stateStart == 0) stateStart = millis();
    if (millis() - stateStart < activeProfile().dwellMs) return;

    Serial.printf("LEADER RELEASED: departing block=%s encounter=%lu\n",
        blockName((BlockId)holdingBlock),
        (unsigned long)holdingEncounterId);
    sendStatus(ST_LEADER_RELEASED);
}
else {
    if (stateStart == 0) stateStart = millis();
    if (millis() - stateStart < activeProfile().dwellMs) return;
}

// Phase 6 — depart
uint8_t departingBlock = myCurrentBlock;
uint32_t departingEnc = holdingEncounterId;

detectState = DETECT_RUNNING;
stopReason = STOP_NONE;

if (ceActive && ceRole == CE_LOCAL) {
    confirmedStopped = false;
    stoppedDebounceActive = false;
    stoppedDebounceStart = 0;
    if (running) rampTarget = CE_LOCAL_PWM;
    return;
}

```

```

// CE Express station stop departure:
// after its every-third-station stop, resume Express service at PWM 120
// without broadcasting a CTO-style DEPARTING message.
if (ceActive && ceRole == CE_EXPRESS) {
    confirmedStopped = false;
    stoppedDebounceActive = false;
    stoppedDebounceStart = 0;
    if (running) rampTarget = CE_EXPRESS_PWM;
    return;
}

resetEncounterState();

sendDeparting(0, departingBlock, departingEnc);
setStatus(ST_DEPARTING);

if (running) rampTarget = cruisePwm;
}

//
=====
=====
// BLOCK DETECTION STATE MACHINE — serviceDetect
//
=====
=====
void serviceDetect() {
    if (millis() < lockoutUntil) return;
    if (millis() - lastSample < SAMPLE_INTERVAL_MS) return;

    lastSample = millis();
    HallState h = readHall();

    if (detectState == DETECT_RUNNING) {
        if (currentHall == HALL_NONE && h != HALL_NONE) {
            firstMagnet = h;
            waitingForClear = true;
            detectState = DETECT_WAIT_SECOND;
            stateStart = millis();
        }
    }
    else if (detectState == DETECT_WAIT_SECOND) {
        if (millis() - stateStart > SECOND_MAGNET_TIMEOUT_MS) {
            waitingForClear = false;
            detectState = DETECT_RUNNING;
        }
        else if (waitingForClear) {

```

```

        if (h == HALL_NONE) waitingForClear = false;
    }
    else if (currentHall == HALL_NONE && h != HALL_NONE) {
        BlockId b = decodeBlock(firstMagnet, h);
        if (b != BLOCK_UNKNOWN) startDwellAtBlock(b);
        lockoutUntil = millis() + PAIR_LOCKOUT_MS;
    }
}
else if (detectState == DETECT_DWELLING) {
    serviceDwelling();
}

currentHall = h;
}

//
=====
=====
// LOCAL SERIAL CONTROL
//
=====
=====
void printLocalStatus() {
    if (trailingSubState != TR_IDLE) {
        // Verbose debug remains stripped in release build.
    }
}

void serviceSerial() {
    if (!Serial.available()) return;

    char c = Serial.read();

    if (c == 'g' || c == 'G') {
        running = true;
        if (detectState == DETECT_RUNNING) rampTarget = cruisePwm;
        Serial.println("LOCAL GO");
    }
    else if (c == 's' || c == 'S') {
        running = false;
        rampTarget = 0;
        Serial.println("LOCAL STOP");
    }
    else if (c == 'p' || c == 'P') {
        printLocalStatus();
    }
    else if (c == 'c' || c == 'C') {

```

```

    clearPeerTable();
    resetEncounterState();
    pendingStartActive = false;
    pendingReleaseActive = false;
    Serial.println("LOCAL CLEAR");
}
}

//
=====
=====
// SETUP
//
=====
=====
void setup() {
    Serial.begin(115200);
    delay(300);
    randomSeed(esp_random());
    analogReadResolution(12);

    mqttSetupTopics();

    ledcAttach(MOTOR_PWM_PIN, PWM_FREQUENCY, PWM_RESOLUTION);
    pinMode(MOTOR_DIR_PIN, OUTPUT);
    direction = DIRECTION_FORWARD;
    applyDirectionPin();
    ledcWrite(MOTOR_PWM_PIN, 0);

    analogSetPinAttenuation(HALL_PIN, ADC_11db);
    pinMode(HALL_PIN, INPUT);
    calibrateBaseline();

    clearPeerTable();
    resetEncounterState();

    // INA219 current/voltage monitor
    Wire.begin(INA_SDA_GPIO, INA_SCL_GPIO);
    delay(200);
    ina219Present = ina219.begin();
    Serial.println(ina219Present
        ? "INA219 OK."
        : "INA219 not found — telemetry unavailable.");

    // WiFi is required for MQTT; ESP-NOW continues to carry the train protocol.
    WiFi.mode(WIFI_STA);
    WiFi.setAutoReconnect(true);

```


